

Industrial Functional Programming ¹

Melinda Tóth, István Bozó



Dept. Programming Languages and Compilers
Eötvös Loránd University, Budapest, Hungary

¹ Supported by TÁMOP-4.1.2.A/1-11/1-2011-0052

Contents

1 NIF

2 Port and Port driver

NIF – Native Implemented Function

- Functions implementation in C/C++
- Compile to `so`, `dll`, load to the Erlang VM
- `gcc -o nif.so -fpic -shared nif.c -I /usr/local/lib/erlang/usr/include/`
- `-on_load(Fun/Arity)`
- `ErlNifFunc: {ErlFunName, Arity, CFunName}`

Specify Load Action and Load NIF

```
-on_load(init/0).
```

```
init()->
```

```
    ok = erlang:load_nif("./nif", init).
```

Implement Dummy Functions

```
dup(_X) ->  
    exit(nif_not_loaded) .
```

```
sq(_Y) ->  
    exit(nif_not_loaded) .
```

Erlang file

```
-module(nif),  
-export([dup/1, sq/1]).  
  
-on_load(init/0).  
  
init()->  
    ok = erlang:load_nif("./nif", init).  
  
dup(_X)->  
    exit(nif_not_loaded).  
  
sq(_Y)->  
    exit(nif_not_loaded).
```

C file

```
#include "erl_nif.h"
extern int dup(int x);
extern int sq(int y);
static ERL_NIF_TERM dup_nif(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[])
{
    int x, ret;
    if (!enif_get_int(env, argv[0], &x)) {
return enif_make_badarg(env);
    }
    ret = x + x;
    return enif_make_int(env, ret);
}
static ERL_NIF_TERM sq_nif(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[])
{
    int y, ret;
    if (!enif_get_int(env, argv[0], &y)) {
return enif_make_badarg(env);
    }
    ret = y * y;
    return enif_make_int(env, ret);
}
static ErlNifFunc nif_funcs[] = {
    {"dup", 1, dup_nif},
    {"sq", 1, sq_nif}
};
ERL_NIF_INIT(nif, nif_funcs, NULL, NULL, NULL, NULL)
```

Define the Functions to Load

```
static ERL_NIF_TERM dup_nif(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[])
{
    int x, ret;
    if (!enif_get_int(env, argv[0], &x)) {
        return enif_make_badarg(env);
    }
    ret = x + x;
    return enif_make_int(env, ret);
}

static ERL_NIF_TERM sq_nif(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[])
{
    int y, ret;
    if (!enif_get_int(env, argv[0], &y)) {
        return enif_make_badarg(env);
    }
    ret = y * y;
    return enif_make_int(env, ret);
}
```


Specify Mappings

```
static ErlNifFunc nif_funcs[] = {  
    {"dup", 1, dup_nif},  
    {"sq", 1, sq_nif}  
};  
ERL_NIF_INIT(nif, nif_funcs, NULL, NULL, NULL, NULL)
```

Port

- Connection with the "world outside"
- "World outside": an OS process
- Receiving and sending messages: standard input/output
- Byte-oriented communication

Port

- `open_port(PortName, PortSettings)`
 `{spawn, Command}, {packet, N}`
- `{Pid, command, Data}` – **send data to port**
- `{Pid, close}, {Pid, {connect, NewPid}}`
- `{Port, {data, Data}}` – **receive data from port**
- `{Port, closed}, {Port, connected},`
 `{'EXIT', Port, Reason}`
- **BIFs:** `port_command/2`, `port_close/1`,
 `port_connect/2`, `ports/0`, `port_info/2`

Port Example: Erlang File

```
-module(port).  
-export([run/3]).  
  
run(Prog, N, M) ->  
  Port = open_port({spawn, Prog}, [{packet, 2}]),  
  Port ! {self(), {command, [N, M]}},  
  receive  
    {Port, {data, [Data]}} ->  
      io:format("~p\n", [Data])  
  end,  
  Port ! {self(), close},  
  receive  
    {Port, closed} ->  
      ok;  
    {'EXIT', Port, Reason} ->  
      {port_terminated, Reason}  
  end.  
end.
```

Port Example: C File

```
#include <stdio.h>

typedef unsigned char byte;

int read_cmd(byte* buf, int cap);
int write_cmd(byte* buf, int len);
int read_exact(byte* buf, int len);
int write_exact(byte* buf, int len);

int read_cmd(byte* buf, int cap)
{
    int len;

    if(read_exact(buf, 2)!=2)
    {
        return -1;
    }

    len = (buf[0]<<8) | buf[1];
    if(len>cap)
    {
        return -1;
    }

    return read_exact(buf, len);
}
```

Port Example: C File

```
int write_cmd(byte* buf, int len)
{
    byte li;

    li = (len >> 8) & 0xff;
    write_exact(&li, 1);

    li = len & 0xff;
    write_exact(&li, 1);

    return write_exact(buf, len);
}

int read_exact(byte* buf, int len)
{
    int i;
    int rd = 0;

    do
    {
        if((i=read(0, buf+rd, len-rd))<0)
        {
            return -1;
        }
        rd += i;
    }
    while(rd<len);

    return len;
}
```

Port Example: C File

```
int write_exact(byte* buf, int len)
{
    int i;
    int wr = 0;

    do
    {
        if((i=write(1, buf+wr, len-wr))<0)
        {
            return -1;
        }
        wr += i;
    }
    while(wr<len);

    return len;
}
```

Port Example: C File

```
int main()
{
    int  len;
    byte buf[100];

    while((len=read_cmd(buf, sizeof(buf)))>0)
    {
        int n = buf[0];
        int m = buf[1];

        int res = 1;

        while(m--) { res *= n; }

        buf[0] = res;
        write_cmd(buf, 1);
    }

    return 1;
}
```


Compiling and Run

- `gcc -o port1.c`
- `erlc port.erl`
- `port:run("port1", 2, 3)`

Connection with other languages

- ErlInterface
- jinterface (Junglerl)
- OTP.NET

On the Next Lecture ...

- Web Servers
- Databases